Kayfabe v3

An architecture to argue with

Stock NVIDIA guest drivers, an emulated device, and a GPU the host keeps using.

STATUS: RESEARCH, 2026-10-27. This is a source audit and a research argument, not a new hardware qualification. The implementation baseline is immutable:

`12c4008705f17033a803e56d4d479bf1275cdb94`

That was `origin/next` when writing began. Experimental results and code outside that baseline carry separate revisions. A moving branch name never upgrades their status.

The guest driver edits its GPU address map in memory that Kayfabe does not intercept. When the driver asks for those edits to take effect, Kayfabe re-reads that map and installs the corresponding host mappings before later work proceeds. The INFERRED hypothesis is that this takes long enough to delay completion until Windows attempts graphics-driver recovery, potentially losing applications' graphics state. Microsoft's default wait for unfinished work to finish or yield after a scheduler request is two seconds; this is not a universal two-second deadline from submission. [1] [77] [78]

The source records contain working Windows and Linux workloads. They do not qualify the pinned baseline on hardware. The later long-wait analysis is UNVERIFIED because its raw trace is not in the repository. Host-plus-guest throughput is UNMEASURED; the coexistence claim has only screenshot evidence. [4] [67]

Reading route: begin with the rules, use the primer for unfamiliar terms, then read Section 6, the limitations and failure case. Section 7 asks questions that can be answered without access to this repository. Numeric citations link to sources; the companion index holds the full source and PR lists. All diagrams are conceptual TikZ drawings, not measured timelines.

Contents

1 The story before the internals **1**

1.1 How to attack the argument . . . 1

2 The product claim and its limits **2**

2.1 Other approaches, without invented comparisons . . . 2

2.2 What would make the title a measured claim? . . . 3

3 How a guest's address-map edit becomes real GPU work **3**

3.1 The update path, end to end . . . 3

3.2 Who manages the resources? . . . 4

3.3 What a channel submits . . . 4

3.4 Dependencies, completion and Windows recovery . . . 5

3.5 Sparse memory is a third state . . . 5

3.6 Three kinds of execution, three meanings of permission . . . 5

4 Architecture v3: where authority and progress live **6**

4.1 Planes and traps . . . 6

4.2 Hostile input and authored operations . . . 6

4.3 Derive, never capture . . . 7

4.4 Seven axes, not one compatibility version . . . 7

4.5 Crate map and the frozen predecessor . . . 8

4.6 Channel kinds and the meaning of completion . . . 8

4.7 Memory store and VA manager . . . 8

4.8 Mirrored scheduling groups . . . 9

4.9 User-ring lifecycle: Scan, Pointer and Window . . . 9

4.10 Lazy observation: a stale view must only delay . . . 10

4.11 Relay, handoff and interrupts . . . 10

4.12 Display and the broker . . . 11

5 Results, denominators and distance from this paper **11**

5.1 Windows positives beside the failure . . . 11

5.2 Hardware cells: what each PASS tests . . . 12

5.3 Windows speed: retain all three denominators . . . 12

5.4 Linux speed: large batches and many small launches . . . 13

5.5 How far the tested code is from the pin . . . 13

5.6 The GPU CI gate and its trust limit . . . 14

6 The uncomfortable part: a plausible explanation still needs a test **15**

6.1 The hypothesis, confidence and falsifier in one place . . . 15

6.2 What is observed, what is reported and what is not inspectable 15
6.3 Earlier failures are not one root cause 16
6.4 A latency budget needs a cost model 16
6.5 Could the design acknowledge early, or avoid the walk? 17
6.6 Sparse coverage trades resource cost against compatibility 17
6.7 Mapping lifetime and a concurrently mutating guest 18
6.8 Pointer forwarding and relay progress still have a proof cost 18
6.9 Per-die coverage, borrowed data and a supported fallback 18
6.10 The host is a victim, and quotas are not isolation 19
6.11 Lifecycle, maintenance and unmeasured scope 20
6.12 Performance evidence must not move with the prose 20
6.13 Licences and release are not settled by a running VM 20
6.14 Test independence and the research stack 21

7 Thirty-six standalone attack questions **22**

A Method and reproducible commands **29**
A.1 Sources and code distance 29
A.2 PR snapshot and historical gap index 30
A.3 Build and verify the publication 30
A.4 Reproducing hardware evidence is separate work 30

B Glossary and historical names **31**
B.1 Decoding old file names 31

C Change record **32**

1 The story before the internals

Kayfabe asks whether a virtual machine can run NVIDIA's ordinary driver while the host keeps its own NVIDIA driver and GPU. It presents a pretend GPU to the guest, then asks the real host driver to do the work. The implementation is research, with no audited security boundary or stable installation interface. Windows needs documented registry settings, but no driver binary patch. [4] [5] [10]

What already works. A dated Windows matrix completed seven of nine workload rows: four graphics-benchmark configurations, two other graphics configurations and Minecraft. One browser-video row failed and one idle row was not run. Linux records include correct large computations close to the host's speed, and slow workloads that issue many small jobs. These results belong to older revisions; *no hardware result presented here tests the paper's pinned baseline*. Section 5 gives their scope and code distance. [52] [4]

The failure this paper tries to explain. A Windows guest can run individual programs, yet pause or reset its graphics driver under mixed application use. Windows has a watchdog, called timeout detection and recovery (TDR). Its default scheduler preemption wait is **two seconds**. If a task cannot finish or yield in that interval, recovery may reset graphics state; applications can lose their graphics device. This is not a rule that every submission must reset exactly two seconds after it was queued. [77] [78]

The proposed explanation, INFERRED. The guest driver edits the GPU's address map in memory the emulator does not intercept. To make an edit effective on the real GPU, Kayfabe must read that map and ask the host driver to install corresponding mappings. If this takes too long and dependent work waits behind it, Windows can see late progress and initiate recovery. That chain is plausible; it is not established as the cause of every reset. [1] [67]

The evidence gap, UNVERIFIED. An analysis note gives a 2.43-second mapping update beside a 2.529-second completion gap. The analysis and raw trace are not in the repository. Those durations motivate a test, not a measured alignment with the watchdog. The decisive experiment must identify the same job, locate the scheduler's preemption request, and separate waiting before submission from execution and notification delay. [67] [68]

The headline limitation, UNMEASURED. Simultaneous host and guest GPU use is shown by the README screenshot, not by a throughput or host-latency experiment. The paper has no measurement of how a busy or faulting guest affects the host's desktop. That is an explicit open question, not an implication of a fast isolated benchmark. [4] [5]

1.1 How to attack the argument

This edition distinguishes six kinds of statement:

MEASURED Executed for this edition, with the command and artifact identified. Here this covers publication checks only: no GPU, VM or rental was used.

REPORTED A repository record gives a result at a named revision. An available table or trace can be checked by the reader; it was not rerun here. Missing raw data stays explicit.

CODE An implementation statement tied to source, not a promise in a design document.

DOC A documented requirement, design or status statement; implementation and acceptance may still be incomplete.

INFERRED An explanation with an observation that could disprove it.

UNVERIFIED A claimed result without inspectable supporting analysis, or an extension beyond the tested evidence. It cannot support an acceptance claim.

Numeric citations link to the source. Full paths, revisions, evidence qualifications and the current

PR snapshot are in the linked companion index, not vocabulary the reader must learn. The code baseline remains `12c4008705f17033a803e56d4d479bf1275cdb94`; later experimental code is always identified separately. [2]

DOC. The claims ratchet, `python3 scripts/ci/debt.py claims`, requires `new=0` for changed documentation. It checks attribution in Markdown and Rust, not the truth of a causal claim and not this TeX prose. A clean result does not certify the paper. Source review and the six labels carry that obligation. [3] [2]

A useful challenge supplies a counterexample, a public contract or a discriminating experiment. For a wake, identify who must wake whom. For a mapping, identify who owns the old bytes and which acknowledgement permits reuse. For speed, specify the workload, sample unit and control. The predecessor records both a probe mistaken for a real workload and a reference model that shared its implementation's mistake. Those incidents justify checking the instruments too. [16]

2 The product claim and its limits

DOC. The guest sees an emulated NVIDIA device. Kayfabe builds host requests itself instead of replaying guest requests. The host driver remains loaded, and its ordinary client interface provides memory and execution resources. Three different permissions matter: the host driver's non-privileged request interface; the GPU channel's USER execution privilege; and the Unix UID of the VMM process. None implies the other two. A scoped non-root run is recorded: the Linux 30-arm suite, the nine GPU gates and a Windows Heaven Direct3D 11 benchmark ran as a throwaway user whose only group was `kvm` (no capabilities), on one Ubuntu 26.04 host with kernel 7.0.0-34, host driver 595.91.07 and an RTX 4070, at revision `0a746117`. The record is a report on the unmerged branch `claude/rootless-audit-20261016` (commit `32d7319b2`, file `traces/rootless\_audit\_20261016/REPORT.txt`); it is not part of this paper's pin. The supplied launch scripts still start as root, and the heavy application lanes and a desktop on the NVIDIA card were not tested. [83] [1] [9]

2.1 Other approaches, without invented comparisons

DOC. VFIO passthrough assigns the physical GPU to one guest, so the host gives it up. NVIDIA vGPU uses a vendor virtualisation stack and its own licensing. API forwarding translates guest driver calls; the sister project nvkvm-pv uses a custom guest module. Kayfabe instead emulates a device so the guest can use a stock NVIDIA driver. The repository has no detailed performance comparison with virtio-gpu, Venus or virgl, and this paper supplies none. [5] [6]

DOC. Public project documentation checked on 2026-10-28 corrects the comparison in the pinned credits. Caraxes serves a synthetic NVIDIA device over vfio-user and retains the host driver. Its quickstart's example run sets `CARAXES_DMA=register`, which needs a binary with `cap_sys_admin` set using sudo; setup also uses sudo for the network device and the guest driver's module options. The same quickstart separately describes submission as an unprivileged host-driver client; this does not make the documented run sudo-free. https://github.com/silasmue/Caraxes/blob/HEAD/docs/vfio/quickstart.md

DOC. Open-IOV's system overview describes its host module binding to the physical GPU and exposing virtual GPUs through mediated devices, with the distribution NVIDIA module unbound. Its install guide requires root or sudo for module loading and device creation, but also describes an emulated path using the distribution driver. These pages do not establish whether the host can keep using the card. https://open-iov.org/index.php/System_Overview https://open-iov.org/index.php/Install_and_First_Run The pinned credits identify gVisor's nvproxy as a source of capability lists, not as evidence of the same stock-device architecture. [6]

2.2 What would make the title a measured claim?

UNMEASURED. A screenshot shows coexistence; it does not grade responsiveness. A convincing shared-use result needs a host workload with retained state, measured alone and under guest load, including guest faults and recovery. Multiple VMs sharing one GPU and resource fairness need different tests again. The security model is a design, not an independent audit. [4] [9]

DOC. Compatibility, deployment permission and distributability are separate questions. The repository's source licence does not grant rights to NVIDIA's driver or Windows. The legal questions in Section 7 include the applicable driver agreement, commercial hosting, datacentre restrictions and any relevant technical restrictions. No benchmark result answers them. [6] [79]

3 How a guest's address-map edit becomes real GPU work

A GPU virtual address means something only in a particular address space. Its page tables translate that address to backing memory and permissions. Kayfabe keeps the virtual names the guest expects, while the host driver builds the real GPU's tables. A *host twin* is the host-side address space or channel created to correspond to a guest object; it is not a copy of the GPU. [1] [23]

3.1 The update path, end to end

DOC. The guest writes page-table entries into GPU memory or guest RAM. Those are ordinary memory stores through untrapped apertures: Kayfabe does not receive a callback for each edit. The guest later requests that translations be made current, through an invalidate register write or the appropriate memory-operation/control path. A register write into BAR0, the register window, traps into the VMM; command methods are handled by channel workers instead. The trap records demand and returns. It does not perform the mapping work. [1] [17] [24]

The *walk* is a traversal of the guest's page tables to discover their current meanings. Kayfabe uses a GPU kernel to read tables already in GPU memory rather than repeatedly pulling them through a CPU view. It compares the discovered mappings with host placements previously confirmed successful. *Apply* means asking the host driver to map and unmap those changed ranges, then invalidate its translation cache. A *reservation* is a host address range whose ownership and geometry constrain where those mappings can be installed. [1] [25] [26]

An acknowledgement says the required mapping transition is settled. Dependent GPU commands must not use an old translation merely because an emulator returned early. A *mapping barrier* is that ordering obligation; a *split* divides a command stream so later work waits for the required refresh. An *admission gate* withholds new software reads or submissions while their mapping dependency is unsettled. These terms describe software ordering, not a global pause of all GPU execution. [17] [68]

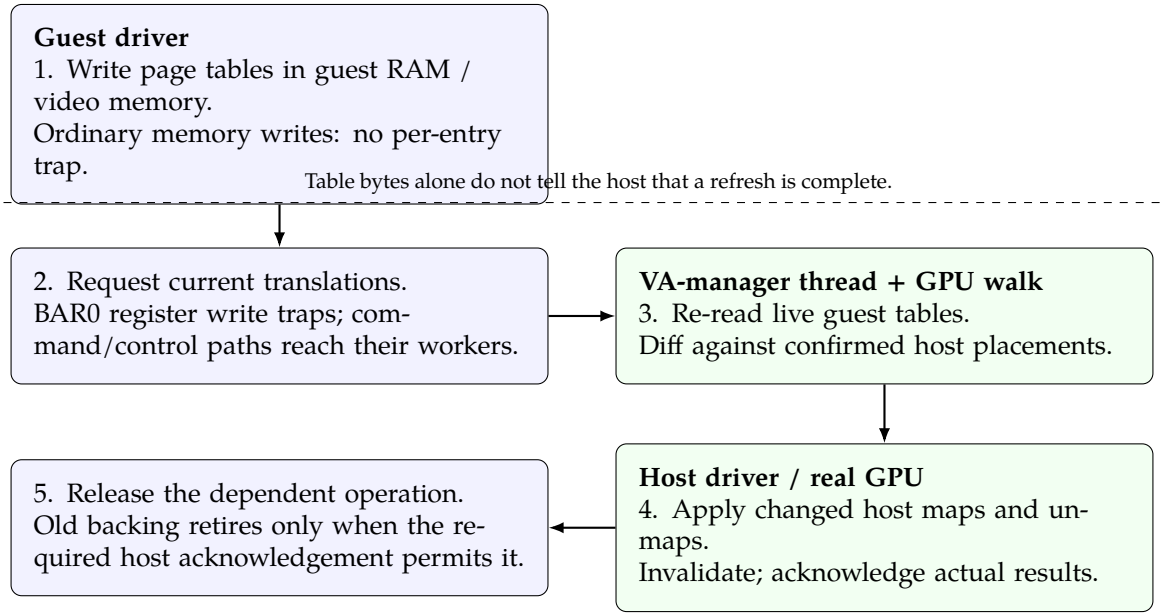


Figure 1: The missing update path. Acknowledging step 2 before step 4 needs another way to prevent stale access; it is not justified by recording the request. [1] [17] [68]

DOC. A refused map can leave an address absent and therefore faulting. A refused unmap can leave old host backing reachable, which is a different risk. The project's rules permit acknowledging the former absence but do not generally permit pretending the latter retirement succeeded. The early-acknowledgement alternative is therefore a real design question: it would need fault/replay or another way to prevent stale access. [13] [26]

3.2 Who manages the resources?

DOC. RM means *resource manager*. The driver allocates objects for memory, address spaces and channels, then names them with handles. A handle is an object name, not a physical address or an access right by itself. On the guest side, the CPU driver speaks to firmware on the GPU System Processor, or GSP. Kayfabe plays that firmware-facing peer, including boot, message queues and replies, then independently asks the real host resource manager for resources. It does not run NVIDIA firmware inside the emulated device. [1] [19]

A page-directory base is the root used to find an address space's tables. The space can outlive that particular root; a recycled root address must not silently identify a new object as the old one. *Mirror* means constructing host mappings from the guest's declared mappings, not handing hostile guest tables directly to the real GPU. [1] [24]

3.3 What a channel submits

A *channel* is an ordered execution context for an engine. A copy engine moves or fills memory; graphics, compute and video engines perform other work. A timeslice group (TSG) groups related channels for scheduling. Preserving each stream's bytes does not by itself preserve its group or shared context. [18] [27]

A channel's *GPFIFO* is a ring of references to command buffers, called pushbuffers. A descriptor gives an address and length; the pointed-to buffer contains method packets and arguments for an engine object. Packets can span descriptors. A software relay that inspects commands must preserve packet state across those boundaries. A *GP reference* is one such command-buffer descriptor,

not the bytes of the commands. [29] [33]

Each channel also has shared producer/consumer state, called USERD. The driver appends ring entries, publishes its write index, GP_PUT, and rings a *doorbell*: a register notification to inspect work. The GPU fetch unit advances GP_GET as it reads entries. GET therefore means "fetched", not "finished". Commands behind a dependency may remain unexecuted after their reference was fetched. A *token* selects a channel route when a doorbell is rung; the write does not prove which guest process sent it. [31] [22]

3.4 Dependencies, completion and Windows recovery

A *semaphore* is a memory value used for dependency ordering: one operation releases a value and another waits for it. A processed marker witnesses that execution crossed a chosen point. A *non-stall interrupt* invites software to inspect completion state; it is not itself a per-packet completion receipt. Host notifications can be engine-wide. [31] [18] [5]

DOC. WDDM, the Windows Display Driver Model, schedules GPU tasks and handles recovery. Microsoft documents a default two-second preemption wait. On a detected timeout Windows attempts recovery; a recovered display can report that the driver stopped responding, while applications may need to recreate their device. Engine and adapter recovery have different scope. A reported 2.43-second host operation is not enough to prove which timeout occurred: the scheduler's request time and the affected task must be identified. [77] [78] [67]

An *Xid* is a host NVIDIA error record; Windows adapter Code 43 is an adapter status; events 153 and 4101 are guest log observations used by the test records. They are different observation domains, not names for one causal defect. A test must correlate them by time and lifetime rather than treating one quiet counter as proof of health. [52] [5]

3.5 Sparse memory is a third state

DOC. A backed mapping points to real storage. A plain-invalid address faults. A sparse mapping intentionally has no backing while absorbing writes and returning zeros on reads. The selective sparse-mapping experiment tries to preserve that distinction; turning every unmapped address sparse would also swallow accesses that should fault. Its finite reservation budget and fallback can therefore affect compatibility as well as resource cost. [65] [64]

DOC. UVM means unified virtual memory. Some managed-memory accesses need a page supplied on first touch and the access replayed. Mirroring mappings already present does not deliver that fault/replay service. This matters both for the remaining application failures and for the proposal to acknowledge a mapping barrier early. [38] [15]

3.6 Three kinds of execution, three meanings of permission

DOC. Kayfabe calls ordinary guest-user execution *Passthrough channels*; this is not VFIO assignment of the whole GPU. Their host twins execute commands under GPU USER privilege in restricted host address spaces. *Translated channels* serve guest-kernel work by rewriting addresses into an emulator-owned privileged space. *Emulated channels* perform work that belongs entirely to the virtual device. Only the last kind completes wholly in software. [18] [1]

GPU USER privilege limits commands on hardware. The host driver's non-privileged request interface limits which host operations Kayfabe may ask for. A Unix UID controls host process credentials. A source or benchmark that establishes one boundary does not establish the other two, and none is a substitute for fault-containment tests. [9] [53]

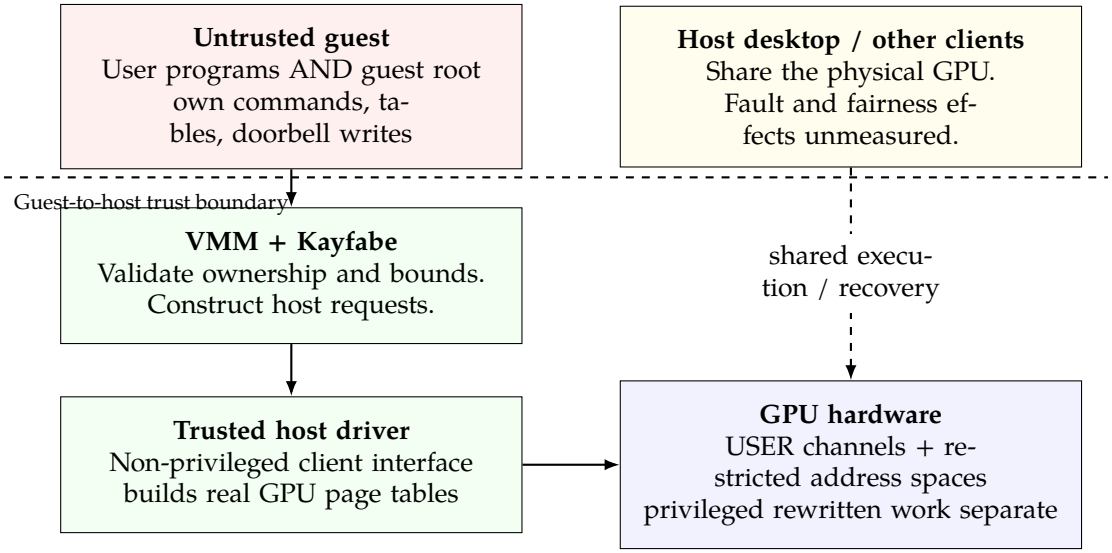


Figure 2: Three permission systems: host request permissions, GPU channel privilege and Unix credentials. Host address isolation alone does not prove host-desktop fault containment. [9] [13]

4 Architecture v3: where authority and progress live

The implementation baseline is the cover revision. CODE statements below name implementation paths; DOC statements describe requirements or published designs. The test descriptions name what could exercise an invariant. Unless a dated result is given in Section 5, their pass state at this baseline is UNVERIFIED: no product test was run for this edition.

4.1 Planes and traps

DOC. The product is the `kf-*` tree linked into the VMM process. A BAR0 write records demand through bounded token and ring state. Workers service channel demand; a register drainer services privileged register operations. BAR1 and BAR2 memory accesses do not trap. There is no separate isolate process or product IPC plane. Confinement applies to the whole hypervisor process. [1] [21] [22]

Invariant: No host mapping or blocking operation runs on a vCPU or under a lock it takes.

Tests and scope: The trap crate's unit tests, core composition tests and source restrictions constrain the reachable trap path. Hardware trap timing checks a separate latency property. [22]

What the test could miss: A nonblocking-looking call can acquire a lock held by slow host work. A mock host that never blocks cannot demonstrate the production lock discipline. The old paper records that failure mode; the restriction must be checked transitively. [16]

4.2 Hostile input and authored operations

DOC. Guest root and guest userspace are untrusted. Host memory operations are constructed from admitted ownership, extent, permission and kind information; guest control messages are not replayed to the host. Host GPU facts come from unprivileged queries or generated open interfaces. This is the intended authority boundary, with the host driver and GPU trusted. [9] [1] [23]

Invariant: No guest-chosen raw address or privilege request creates new host authority.

Tests and scope: Hostile range/ownership tests, channel privilege tests and the physical-operand hardware oracle exercise different pieces of this claim. [9] [53]

What the test could miss: Checking a decoder cannot establish that every caller uses it. A guest-declared OS identity also cannot prove a safe mapping scope. Whole-path reachability and multiple-VM fault effects remain separate review obligations. [13] [9]

4.3 Derive, never capture

DOC. The architecture prefers facts read from the host's own unprivileged interface, computed virtual-device state and rules generated from pinned open headers. A device identity is not permission to copy another die's answer. Request validation and field-level virtual support still apply when an answer comes from the host. Published measured exceptions belong to exact die/driver coverage, not an invisible default table. [1] [40]

Invariant: Every guest-visible field has a source and a defined unsupported case; an unknown die never silently inherits a neighbour's observation.

Tests and scope: The constants strategy requires source tags, derivation oracles, wrong-answer mutations and same-die native acceptance. It explicitly separates the implemented baseline from table consumers on branches. [40] [41]

What the test could miss: A source tag proves provenance only if its source actually supports the field. A boot that never exercises an optional feature cannot verify that feature, and agreement between a few dies is not a mechanism for a family rule. [40]

4.4 Seven axes, not one compatibility version

DOC. The design separates guest kernel, guest driver, host driver, GPU architecture, GPU die, VMM and guest OS. An architecture supplies family mechanisms; a die supplies actual inventory and capability facts. The guest ABI controls decoding; the host ABI controls authored calls. The VMM supplies memory, interrupts and lifecycle events. These are distinct axes even when a test holds several fixed. [12] [40] [39]

Axis	Boundary to inspect	Counterexample worth testing
Guest kernel	DMA addressing / virtual IOMMU	IOVA mistaken for guest physical address
Guest driver	versioned decode / accepted replies	layout changes within a branch
Host driver	authored RM ABI	successful syscall with failed in-band status
Architecture	class / register / MMU family	different engine or boot protocol
Die	runtime facts / measured exceptions	floor-swept or unmeasured die
VMM	memory, IRQ, reset adapter	reset origin or wake semantics differ
Guest OS	driver protocol and scheduling	Windows registration / recovery path

The counterexamples are proposed attacks, not claims that each has occurred at the baseline. A matrix that changes one axis at a time is evidence for those cells, not the full Cartesian product. [39] [40]

Invariant: Guest and host ABI choices remain independent, and OS or VMM identity never grants extra memory authority.

Tests and scope: The driver matrix tests named host/guest cells; versioned decoder oracles check layouts. Portability beyond QEMU and x86-64 still needs real adapter evidence. [39] [11]

What the test could miss: A layout may change within a driver branch, and a cross-build cannot exercise mapping, interrupt or reset semantics. Passing one axis with the others fixed is not the full grid.

4.5 Crate map and the frozen predecessor

DOC. The following is a functional map from `ARCHITECTURE.md`, not a generated claim about every dependency edge. [1]

Crates	Responsibility
`kf-trap`, `kf-core`	bounded demand publication and plane composition
`kf-gsp`, `kf-rm`	firmware-facing protocol and resource-manager model
`kf-chip`, `kf-abi`	host facts, family rules and versioned generated layouts
`kf-host`	one in-process host RM session and authored verbs
`kf-mem`, `kf-cuda`	backing store, VA manager, GPU walk and diff protocol
`kf-chan`	channel birth, translated work, relay and completions
`kf-qemu` + C device	QEMU memory, lifecycle, interrupt and device adapter
`kf-disp`, `kf-broker`	emulated display and VMM-neutral broker protocol
`kf-oprom`, `kf-gop-image`	generated boot-display container and build
`kf-harness`	real-GPU tests without QEMU
`kf-census`, `kf-crec`, `kf-trace`	offline tools and historical oracles
`kf-arch`, `kf-linux-raw`, `kf-util`	vocabulary, audited OS calls, utilities

DOC. The older `kayfabe-*` crates remain frozen; the raw-client grader depends on part of that tree. Product `kf-*` crates do not depend on it. Unsafe code is confined to named adapter crates and `*_unsafe.rs` files. A crate inventory is a source-level fact; it says nothing about whether a feature has a production caller. [2] [1]

Invariant: Product crates retain their dependency and unsafe boundaries; the frozen grader tree never becomes a hidden product dependency.

Tests and scope: Workspace manifests and CI boundary checks enforce structural restrictions. Reached-path oracles and real adapter tests address behaviour separately. [2] [1]

What the test could miss: A legal dependency graph can still contain an unused implementation or a production-only blocking path. Source structure does not establish reachability or hardware correctness.

4.6 Channel kinds and the meaning of completion

DOC. Passthrough creates a host twin for user work and lets the real engine execute its commands. Translated channels rewrite kernel copy work into an owned address window and submit it to the real engine. Emulated channels execute only work whose meaning belongs entirely to the VMM. The three kinds differ in authority and completion, not simply speed. [18]

Invariant: No CPU executor substitutes for GPU work; no completion is forged for work submitted to hardware.

Tests and scope: The channel suites and real-engine gates check actual execution, content and notification. The translated-plane design separates address arithmetic from execution. [17]

What the test could miss: An application can report success without checking its GPU error or result. A harness must validate the output and completion path, not merely the process exit. [15]

4.7 Memory store and VA manager

DOC. Guest video memory is one host allocation with offsets corresponding to guest frame-buffer offsets. Guest system memory comes from the VMM's shared backing. A GPU walk reads

guest tables and diffs them against confirmed host placements. Host maps/unmaps apply the changes, invalidate, and acknowledge runs. A refused map remains a difference. The committed-placement record is not an adopted guest page table. [1] [25] [24]

Invariant: Only acknowledged host placements become committed state. Old backing remains owned until the relevant invalidation acknowledges its retirement.

Tests and scope: The diff model, apply tests and hardware walk gate compare emitted runs, acknowledgements and resulting translations. The retirement experiment adds explicit delayed-free tests. [25] [26] [80]

What the test could miss: A software model can assume host RM ordering that hardware does not provide. Table roots, shared subtrees, root withdrawal and failed host invalidation require separate schedules. Whole-space scanning is not equivalent to remapping every unchanged leaf. [68]

4.8 Mirrored scheduling groups

CODE. Host group/member construction preserves a guest TSG grouping, and twin lifetime follows the trusted object model. The late-joiner design documents why a newly added channel cannot silently miss earlier scheduling state. Address-space mirroring and TSG mirroring are separate concerns: preserving one does not prove the other. [27] [28] [18]

Invariant: Channels in one guest scheduling group retain the corresponding host grouping and required state as members arrive and leave.

Tests and scope: Grouping tests and the video-engine validation exercise group creation and related channel use. [27] [37]

What the test could miss: A test with all channels present at initial configuration misses late arrival. Reused handles and reset generations can make a correct group lookup refer to the wrong life. [28]

4.9 User-ring lifecycle: Scan, Pointer and Window

DOC. A registration is a guest request that arms a later software action in its command stream; that action can move the ring source. Windows user-work can therefore require software intervention around registration and ring migration. In Scan mode the worker inspects command packets. Pointer mode forwards admitted GP references without reading the pushbuffer. Window mode restores scanning across the registration boundary. An anchor is an observed producer position selected as that boundary. The clean anchor must have been observed before the registration reply; a later convenient cursor cannot retroactively establish that boundary. [30]

Invariant: Doorbell-covered entries appear once, in order and bit-exact; Pointer reads no pushbuffer; a validated descriptor is the snapshot actually published.

Tests and scope: Real-turn model, property and threaded stress suites cover mode transitions, ring migration, packet continuation, poisoned bytes above PUT and registration at different positions. [30] [33]

What the test could miss: Finite model schedules can omit a real host lock holder, DMA ordering or guest mutation. A normal-user experiment cannot certify a hostile user that rewrites its own memory. The boundary remains native USER privilege plus admitted mappings. [13]

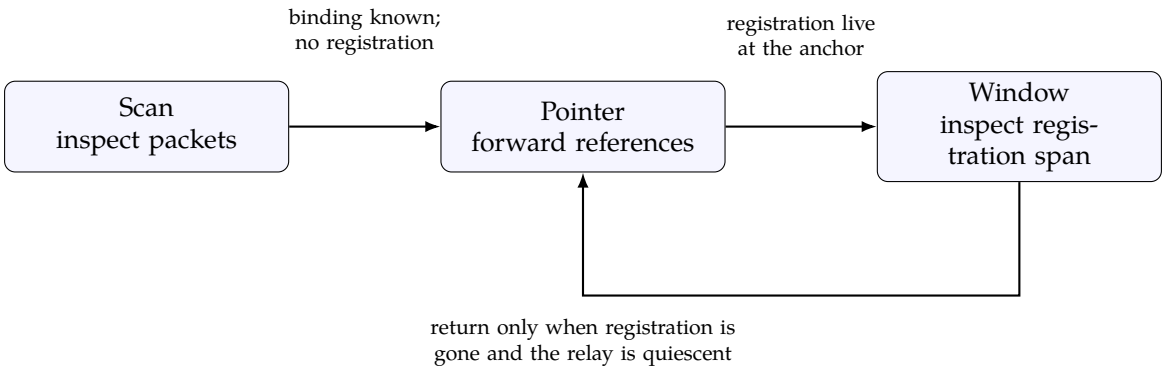


Figure 3: Main lifecycle path; direct Scan-to-Window and teardown edges are omitted. Registration and packet-boundary rules govern each transition. [30]

4.10 Lazy observation: a stale view must only delay

CODE. Pointer mode caches a previously loaded PUT within the same ring generation and reduces private USERD observations. Scan and Window remain fresh. Low credits, waits, split segments and a full guest ring force observations. A stale GET may reduce available credit; it must never increase credit or release a marker gate early. Mode changes and migration invalidate cached state. [30] [32]

Invariant: No idle decision relies on cached PUT, and no stale observation is the sole missing wake for a wait that can already be satisfied.

Tests and scope: The lazy-turn suites compare forwarding identity with observation enabled and disabled, exercise small rings, and test forced observations and generation changes. [30]

What the test could miss: A counter of fewer reads is not proof of liveness. Combined performance arms cannot attribute speed to each optimisation separately. Hardware completion and holder-release timing remain outside the pure predicate model. [30]

4.11 Relay, handoff and interrupts

CODE. The relay has explicit progress obligations. If a storage, row or page-table holder prevents progress, release must arrange another attempt; repeatedly trying a busy lock is not an event-delivery proof. The baseline includes the relay handoff fix cited by the README and a later merged lost-wake correction. Experimental split-latency instrumentation keeps that relay design rather than introducing an attempt limit. [4] [33] [68]

Invariant: Registering a waiter concurrently with completion or lock release cannot lose the last wake. An interrupt invites a check; it does not manufacture a completed fence.

Tests and scope: Relay interleaving tests, completion tests and the live regressions cited in the README exercise parts of this claim. The later probe experiment explicitly diagnoses missing wakes. [4] [82]

What the test could miss: Host events are GPU-wide, so an interrupt can be unrelated to this channel. Observing a marker late does not timestamp when the GPU wrote it. A worker busy elsewhere can delay observation without a lost physical event. [5] [68]

4.12 Display and the broker

DOC. An emulated display engine accepts the stock driver's display protocol; a separate broker presents owned frame copies and handles input. The display contract requires guest flips to complete according to guest semantics while copies select the newest frame independently. The broker owns cursor composition, including inversion. Its absence or silence must not freeze guest display progress. [1] [34] [13]

Invariant: Guest flip completion is independent of a stalled viewer; frames and credits remain bounded; the broker never receives arbitrary guest memory as an owned frame.

Tests and scope: Display engine tests, plane-output oracles, socket simulations and pacing models exercise flip order, slow brokers and pixel formats. [34] [35] [36]

What the test could miss: A fake broker cannot establish compositor, DMA-buffer or colour behaviour on every desktop. Correct pixels do not prove correct flip latency, and a correct cursor does not prove safe frame ownership. [36] [9]

5 Results, denominators and distance from this paper

No hardware row below is a test of the pinned baseline. These are dated repository records, not new runs. Their product-path distance is given below. A result's revision and sample unit remain part of its meaning even when the topic has since been merged.

5.1 Windows positives beside the failure

REPORTED. On the physical RTX 4070 AD104 host, host driver 595.91.07 and Windows guest 580.88, the 2026-10-14 matrix at `eb9f2aa3` has **seven PASS rows of nine**: Heaven Direct3D 11 windowed and fullscreen; Heaven OpenGL windowed and fullscreen; GravityMark Direct3D 12; GravityMark Vulkan; and Minecraft. This is seven workload configurations, not seven distinct applications. Each has one recorded row, not a repeated-boot confidence interval. [52]

PASS means the application's own completion plus the matrix's zero-error conditions: no host Xid attributed to this VMM, no firmware restart during the application, and no listed guest driver/display error. It is not a numerical-output oracle for every application. Edge video is FAIL, with recovery after about 7.3 minutes; idle is NOT RUN. Heaven's Direct3D 11 rows completed at about **18.0 and 21.2 fps**: a correctness-oriented PASS can coexist with poor speed. These settings differ from the later performance table. [52] [58]

REPORTED. A later README record describes more than fifteen minutes of Minecraft beside Heaven at `89fc8a222` without the earlier freeze. It does not repeat the full matrix or qualify the later sparse-mapping experiment. The stalled-pair comparison printed beside it, **0/26 after versus 3/12 before**, is explicitly a *Linux* relay test result, not a Windows result and not the Enscape workload. Its raw pair-by-pair analysis is not indexed here, so the comparison remains a reported result. [4]

UNMEASURED. The coexistence screenshot establishes no host-plus-guest throughput, host UI latency or live-host fault-containment metric. All isolated results below leave that headline question open. [4] [5]

5.2 Hardware cells: what each PASS tests

Cell	Runtime revision	Reported scope and sample unit
Linux / TU116	`1915bd71`	Gates 9/9; native and thin guest 30/30. One merge-bar campaign; ladder 4/4 at `a295e2ce`, documented as the same code. [55]
Linux / AD104 workstation	`fc2dadc4`	Gates 9/9; native and thin 30/30. One campaign; ladder 4/4 at `d40dafbf`. [56]
Linux / GB205	`d40dafbf`	Gates 9/9; native and thin 30/30; ladder 4/4. One cell, not a Windows pass. [56]
Linux / GA104	`714ec29d3`	595.91.07 host and guest; gates 9/9 plus gate 10; native and thin 30/30; one complete qualification. This runtime is on a divergent branch. [54]

REPORTED. The September family cells used nested rented hosts and 580.159.04 open drivers. A 30-arm thin suite boots an arm at a time; 30/30 is not thirty repeated mixed-application boots. The later GA104 record explicitly corrected a provider's "RTX 3060" marketing label after reading its hardware identity. This paper therefore does not infer a die from that marketing name in the older performance records. [55] [56] [54]

REPORTED. The earlier Linux application record reports 61/65 applications and six stream probes at `2830988f`, and 100 sequential processes in one boot *without* persistence mode at `4c48ca0c`, on a nested rented RTX 3060 with 580.159.04 host and guest drivers. Its exact die is not re-established here. Four failures require managed-memory demand paging. A graphics record at `d06833f0` reports 38/38 items, 31 with byte-identical output, on a GA104. These test content and workload coverage; neither is baseline-wide acceptance. [15] [51] [37]

DOC. The status document also reports 3,138 GPU-free tests passing at its October 15 integration. That dated count is not a current test result. This edition ran no Cargo command and gives no pass count or GPU-gate verdict for the cover revision. [15]

5.3 Windows speed: retain all three denominators

The following concerns Heaven 4.0, Direct3D 11, 1920×1080 windowed, Ultra, about 260 seconds per timed run, on the *same physical, non-nested RTX 4070 AD104 host*. The Windows guest uses 580.88; Kayfabe's host uses 595.91.07. Native means the Windows guest owns the physical GPU through VFIO, so it does not measure simultaneous host use. [69] [58]

Arm	FPS / native fraction	Sample and provenance
VFIO native	330.05 median [329.48, 330.18]	$n = 3$, 2026-10-15; QEMU `666ea966` without the emulated device. Not repeated with every later arm. [70]
Earlier code, scan on, before the port	about 89–102 / 27–31%	The later control summary pools samples at different revisions and host states; no pooled n is inferred. An inspectable `0a7461173` subset is $n = 3$: median 97.65, range 96.04–102.44. That revision does not contain the lazy-observation commits. [58] [70]
Research build before the port: scan off after setup + lazy reads	about 296.5–299.2 / 90–91%	Three public FPS records from 2026-10-16/17, measured on a research build. The stated runtime revision does not resolve, so its runtime provenance is UNVERIFIED. Heaven only. [71] [72] [73]

DOC. The two mechanisms in the last row were later ported. Pointer forwarding and lazy observation are on master and default on in Pointer mode; Scan and Window still inspect fresh

state. [32] [30] The 296–299 fps records were measured on the earlier research build, not on master, and speed on current master needs a new measurement, which is pending. The unresolved runtime hash is a separate problem from this merge status.

REPORTED. The research-build FPS values are inspectable in committed summaries. They do not supply a reproducible source runtime or an integrated acceptance pass. The research build skipped ordinary command-buffer inspection after setup and combined two observation optimisations; it cannot attribute their gains separately. The fractions above are arithmetic against the 330.05 reference, not new measurements. Different quality settings, host pressure or window sizes change the denominator; the 18–21 fps matrix rows must not be silently mixed into this table. [71] [58] [52]

5.4 Linux speed: large batches and many small launches

REPORTED. On a rented nested RTX 3060 box with 580.159.04, a 2048-square matrix multiply at `b5c9f717` gave **784 versus 786 GFLOPS**, ratio **0.997**. The CUDA ladder record says one repetition; it supplies no between-boot timing spread. Its product claim is content-checked large work, not launch latency. [74]

REPORTED. At `9d82f259`, the same class of nested RTX 3060 test host gave llama.cpp prompt processing at **0.998** of host speed and token generation at **0.877**. The committed logs specify two internal repetitions per benchmark ($n = 2$ within each recorded invocation); this is one application campaign, not multiple independent boots. These batch-heavy results should be read beside eager decoding rather than replaced by it. [75] [76] [57]

REPORTED. For eager Qwen2-0.5B decoding at `e5ff45ff`, the nested AMD EPYC 7452 / RTX 3060 record, using 580.159.04, gives for 512-token warm decode guest **12.62 tokens/s [12.40, 12.94],** $n = 3$ and same-box host **43.98 [43.59, 44.63],** $n = 3$; ratio **0.287**, about **0.29**. Its third reference lane on another Xeon box is **28.28 [27.96, 28.84],** $n = 3$; it is not the same-box denominator. The record identifies output equality separately. The old die label is not independently confirmed for this campaign. [59] [60]

INFERRED. The workload shape explains why both favourable and poor ratios matter. A large batch amortises submission overhead; eager code repeatedly pays it. Nesting adds another virtualisation layer to the rented host, so its guest/host ratio cannot predict physical-host performance without a matched experiment. Placement in GPU memory versus host memory, clock state, and the control's own variability also remain part of the experiment. [57] [16]

5.5 How far the tested code is from the pin

MEASURED for this edition, Git comparison at the cover revision. The table counts changed paths under `crates/`, `qemu/` and `cuda/`. Commit counts are those reachable only from the pin / only from the result. Zero in the second column means ancestor; nonzero means divergence. These are code distances, not an estimate of failure probability. All rows' hardware status at the pin is **NOT RERUN in this edition**; no later receipt indexed here certifies these complete results at that pin.

Result	Runtime	Paths	Pin-only	Result-only
TU116 merge bar	`1915bd71`	442	1987	0
TU116 ladder	`a295e2ce`	442	1988	0
AD104 merge bar	`fc2dadc4`	442	1982	0
GB205 bar / both ladders	`d40dafbf`	442	1985	0
GA104 candidate	`714ec29d3`	64	629	52
Windows matrix	`eb9f2aa3`	135	864	0

Result	Runtime	Paths	Pin-only	Result-only
Mixed Windows session	`89fc8a222`	84	660	0
Linux 61/65 apps	`2830988f`	441	1984	0
Linux 100 processes	`4c48ca0c`	639	2114	0
Linux headless graphics	`d06833f0`	641	2197	0
VFIO native control (QEMU)	`666ea966`	132	822	0
Earlier-code FPS subset (before the port)	`0a7461173`	130	821	0
Linux matmul	`b5c9f717`	307	1518	0
Linux llama.cpp	`9d82f259`	382	1754	0
Linux eager decoding	`e5ff45ff`	801	2485	0
Linux suspend/resume	`559db30c`	169	938	0
Host-call cost study	`6d95870c7`	150	629	162
Supplied long-wait analysis	`64c50ad4`	135	35	182
Research-build Heaven (before the port)	unknown	—	—	—

A merged fix can be present while its old workload receipt remains distant. Conversely, a result on a divergent candidate cannot qualify the baseline merely because it is called “production” in an old record. Reusing evidence needs review of changed inputs or a fresh exact-tree run; neither changed-file counts nor elapsed time performs that review.

5.6 The GPU CI gate and its trust limit

DOC. A Cloudflare Worker authorises a GitHub workflow/run attempt and tested commit, while a Durable Object ledger records rentals, deadlines and reserved spending. The provider key stays outside the runner and rented machine. The rented operator and its reports are untrusted; a valid callback authenticates a run capability, not honest GPU execution. The result is a regression tripwire, not release provenance or a security audit. [49] [50]

DOC. The October 27 policy sends new PRs to `next`, requiring CI there and CI plus the GPU check on a final batch to `master`. Older PRs retain feature-branch or master bases; those stacks predate this rule. The refreshed companion snapshot records every base and known parent PR rather than implying the current policy describes all historical PRs. [2]

Invariant: Admission binds the approved SHA and cleanup acts on ledger-owned instances.

Tests and scope: Broker tests and recorded provider campaigns target identity, accounting, result binding, retries and cleanup. Their status belongs to their own revisions. [49] [50]

What the test could miss: A fake Worker or provider can report a PASS without executing the proposed code. Node doubles also miss platform-specific fetch behaviour. Neither is a release-attestation mechanism.

6 The uncomfortable part: a plausible explanation still needs a test

6.1 The hypothesis, confidence and falsifier in one place

> **INFERRED hypothesis:** making a guest address-map edit effective on the host delays work that depends on it; completion then arrives too late for Windows' recovery machinery. **Documented reference:** the default scheduler preemption wait is two seconds, not two seconds from every submission. **UNVERIFIED analysis:** one note places a 2.43-second apply beside a 2.529-second completion gap, but the analysis is not in the repository. **Falsifier:** identify the same affected task and show that its long gap occurred after host publication with no corresponding mapping wait, or that it had already completed while only observation or guest notification was late. [77] [67] [68]

The later campaign uses the Enscape rendering workload in a graphics benchmark sequence. It is a workload name, not an internal subsystem. No SPEC score or proprietary workload image is needed for the failure argument. Individual Windows workloads did complete in the earlier matrix, as Section 5 shows; the question is whether mixed use stays correct and responsive with the full mapping configuration. [63] [52]

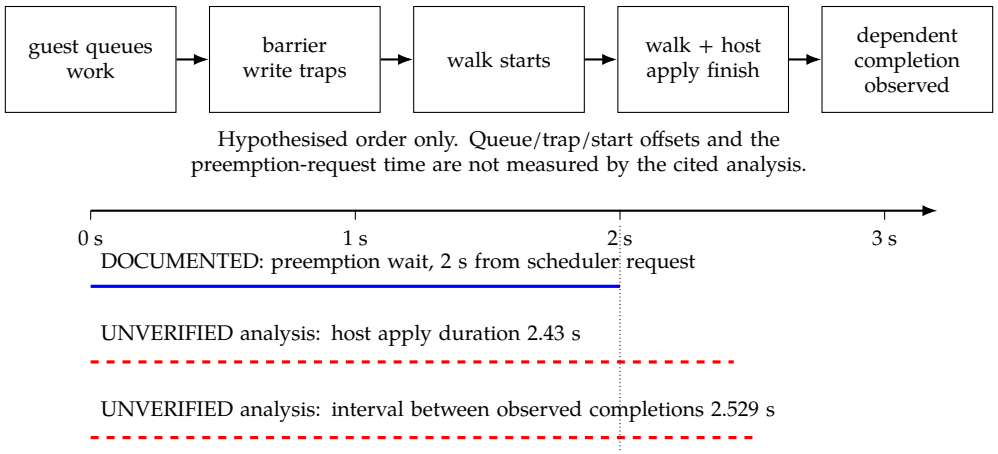


Figure 4: Duration comparison, not a measured shared clock. The lower bars have independent origins: the 2 s marker cannot locate an actual reset without the preemption request and packet identity. The long-duration analysis is not in the repository. [77] [67]

6.2 What is observed, what is reported and what is not inspectable

REPORTED. The earlier Windows matrix contains seven completed workload configurations and an Edge recovery failure. A later README session reports a clean mixed Minecraft/Heaven interval. These are different workloads and revisions from the later sparse-mapping case; neither makes the later case impossible. [52] [4]

UNVERIFIED. An experimental note attributes roughly **93–94%** of mapping walks in two evidence-set boots to one privileged copy channel. Its supporting memo and producing analysis are not in the repository. The exact count pairs and boot identifiers are confined to the companion's evidence qualification, labelled UNVERIFIED. The proportion is not a measured attribution in this paper. [68]

UNVERIFIED. The **2.43 s / 2.529 s** pair and a longer-watchdog arm without a recovery before its time limit are likewise supplied analysis without a public raw trace. They support a question about lateness, not the conclusion that completion was proven late rather than lost. Matching

periodic walk and completion intervals does not establish packet identity, clock origin or when the GPU actually finished. [67]

DOC. The experimental implementation records native publication and first CPU observation of a marker. It explicitly lacks a GPU timestamp for the marker write. Publication-to-observed marker therefore combines hardware execution and observation delay. A worker stuck elsewhere can make an already finished operation look slow. An engine-wide interrupt can also precede observation and refer to unrelated work. [68]

Acceptance gap: none of the indexed receipts establishes a full-profile, default-watchdog, sparse-mapping-enabled Windows pass for the failing mixed workload. The implementation notes retain pending hardware acceptance and an off-by-default experiment. A longer observation window, a passing CPU model or a fast isolated renderer cannot fill this gap. This is a limit of the indexed evidence, not proof that no unrecorded run exists. [65] [64] [68]

6.3 Earlier failures are not one root cause

An uncleared translation trigger. DOC. A historical Windows failure left the guest waiting for an invalidate acknowledgement. The project's ruling permits completion over absent guest leaves, while keeping refused host unmap or invalidate distinct. A later reservation-edge defect was the emulator's mapping failure, not malformed guest tables. This is a documented predecessor, not evidence for every later reset. [13]

A fetch cursor used as an execution witness. DOC. The event-loop investigation found that GET could advance beyond a dependency before the following work ran. A processed reference replaced that proof. "Fetched, therefore finished" was too strong. [31]

A relay wake with no remaining holder. DOC. The consumed-ring contention record describes a real protocol defect: a worker used a notification, failed to acquire a busy slot, and could leave no pending wake. Behavioural tests exposed cases that source-shape guards missed. Its recovery does not prove that every Windows reset was a lost wake. [48]

Expensive failed sparse admission. DOC. The first implementation could clone and restore the whole reservation tree on failure, return an uncounted hard error, and retry the same failed acknowledgement. Later work introduced local undo and explicit decline tracking. These code mechanisms are independently inspectable; the claimed Windows timing decomposition remains unverified without its analysis. [62] [63]

The mapping gate alone blocks observation. DOC. Tests documented on the original gate still harvested a published marker and wrote GET before the PUT admission check. They contradict that narrow explanation. They do not establish timely scheduling, notification, or an absence of waiting before publication. [67]

A missing wake and excessive mapping work can both exist. Fixing either does not eliminate the other, and a failure that appears after an optimisation is not necessarily caused by that optimisation. A useful chronology preserves the workload, changed variable and rejected prediction, rather than reusing a broad name such as "the Windows hang".

6.4 A latency budget needs a cost model

For a strictly serial refresh, a useful accounting model is

$$T_{\text{barrier}} = T_{\text{queue}} + T_{\text{walk}}(P) + \sum_{i=1}^{M} T_{\text{map},i} + \sum_{j=1}^{R} T_{\text{reservation},j} + T_{\text{repair}} + T_{\text{invalidate}} + T_{\text{release}}.$$

This is an **INFERRED measurement plan**, not a fitted or measured bound. P is the table work actually visited, M the ordinary map/unmap calls and R the reservation-layout calls. The stages must be timed without overlap before they can be added. Shared batches must not be charged once per waiting ticket and then summed as if they were independent work. [68] [63]

DOC. Finite object and walker caps bound storage or admitted work. They do not give a wall-time bound for a native driver call. A slice allowance bounds optional work that may start; already started replacement may need several calls to repair ownership, and ordinary mapping or teardown can lie outside that allowance. A claimed 100-ms slice is therefore not an upper bound on every barrier. [63] [64] [67]

UNMEASURED. There is no established worst-case barrier bound, expressed in guest layout and host load, for the cover revision. Nor is there a measured bound on possible serialization inside the shared host driver. The most important victim may be the host's own desktop, not another guest. Those missing bounds determine whether benign large workloads or a hostile layout can deny useful service. [9] [68]

6.5 Could the design acknowledge early, or avoid the walk?

INFERRED. Early acknowledgement would remove a CPU-visible wait only if later GPU accesses remain correct. If a stale host mapping still points to old memory, the access may succeed with wrong data instead of faulting. Fault/replay can help only where a replayable fault path actually exists and the old mapping has been safely removed. The stock path does not provide the general managed-memory demand-fault service. [38] [26]

A second possible architecture would let hardware use validated guest page tables directly, or a separately isolated translation context. This paper claims no such host interface exists for the current implementation. The open question is whether a public, unprivileged host interface could preserve privilege, ownership, revocation and hostile mutation while deleting the software walk. A speculative mention of hardware translation facilities is not an implementation plan or an acceptance result.

DOC. A split currently asks for a fresh post-admission walk so later commands do not inherit an older snapshot. Tickets already pending for one target can share a submission. A channel that waits for each answer before requesting the next supplies no concurrent work to coalesce. Cheaper batching can therefore leave the serial bottleneck intact. [68]

6.6 Sparse coverage trades resource cost against compatibility

DOC. Selective sparse reservations preserve the difference between absorbed sparse access and a faulting invalid address. A blanket sparse address space would weaken the negative controls. The experiment stays off by default; a small native oracle is not a full Windows workload pass. [65] [81]

DOC. Finite budgets and permanent host-owned exclusions protect host resources. A declined sparse request can nevertheless leave the guest with invalid coverage where it expected sparse semantics. Counting that deficit is necessary, but does not make the guest compatible. On interval-tracker overflow, a conservative upper bound must remain visibly uncertain rather than becoming a false zero. [64]

DOC. The latest experimental continuation moves optional geometry out of the mandatory split release path and rotates target identities. Mandatory repair still completes. Under continuous mandatory work, optional coverage can remain unhonoured. The design has not established a hard total-apply deadline or a Windows cure. [68]

INFERRED. Increasing the budget alone could hide the first capacity cliff while increasing memory and teardown cost. A better representation might coalesce live sparse coverage, but release-

before-reserve can create a forbidden hole in unchanged memory. The useful question is which native replacement primitive, if any, preserves continuity without stopping the GPU. [62] [64]

6.7 Mapping lifetime and a concurrently mutating guest

DOC. Mirrored tables create two obligations: a new mapping must have the intended meaning, and the old backing must remain owned while hardware may still use it. The retirement proposal delays free until host invalidation acknowledges. Failure retains ownership rather than fabricating a successful retirement. [80]

DOC. A live GPU walk is not an atomic snapshot of guest memory. The rulings record an example where a recycled table was read at the wrong level during a transition; an apparent malformed leaf was not a stable declaration. That illustrates a time-of-check/time-of-use problem, not proof that the guest has stopped changing its tables. [13]

INFERRED. A safety argument must specify allowed observations during change. The current experimental contract permits old backing, new backing, sparse absorption or a fault for a changed address, while unchanged addresses stay accessible. A host-target gate does not certify a range unchanged; a bounded range certificate would also need root generations, aliases, publication revalidation and an explicit overflow fallback. [65] [67] [68]

DOC. Narrowing by the host object being mutated avoids an assumption that distinct guest roots cannot share table pages. It also cannot remove shared-engine contention or a host-driver-wide serialization point. Those are different dependencies that need different observations. [67]

6.8 Pointer forwarding and relay progress still have a proof cost

DOC. Pointer mode avoids inspecting ordinary command bytes after setup; a live registration restores scanning over the required boundary. Its anchor must be an observed producer position before the registration reply. Choosing a convenient later cursor would change the proof. The native USER channel and its restricted mappings still carry execution authority. [30] [13]

DOC. Lazy observation saves reads but delays guest-visible GET. Small rings, waits, low credits and mode transitions must force enough observation to prevent false capacity or a missed wake. A large benchmark ring does not exercise the worst backpressure case. The fast scratch runs combined two changes, so their separate benefit remains unmeasured. [30] [58]

DOC. The repository also contains a retry-set relay alternative: recheck outstanding state on every loop and spin when no dependable wake source exists. It explores returning slot credit at software-read progress rather than hardware-fetch progress. Its checkpoint records unresolved model expectations and no hardware run. Simpler loop structure is a candidate to test, not a result that the new semantics are correct. [61]

A release notification, doorbell or interrupt can be only a hint if durable state still preserves the work. If a worker consumes the final hint and parks while work remains, a last-resort timer may hide the defect. A holder/waiter protocol must survive release before, during and after waiter registration. Section 7 poses that as a standalone concurrency problem. [48] [2]

6.9 Per-die coverage, borrowed data and a supported fallback

DOC. The project's public source rule prefers computed virtual state and pinned open interfaces, then safe host queries, then exact-die/driver measurements where necessary. A family rule needs both observed agreement and a mechanism. An unmeasured die gets a defined refusal or reduced mode, not a neighbouring die's unlabelled row. [40] [41]

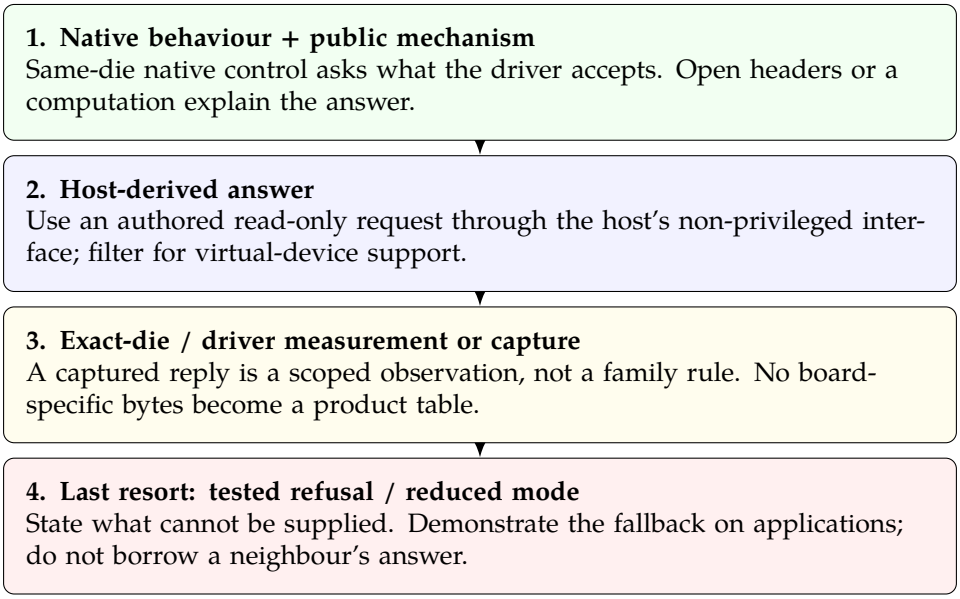


Figure 5: When a higher route cannot supply an answer, record why before moving down. Native observation is an oracle, not authority to reuse its bytes on another die. Closed-binary material is not a public implementation source. [40] [41]

DOC. The requirements describe borrowed display pages for some Windows dies. This paper has not established those pages' measurement origin; they are compatibility/provenance debt, not a clean implementation of derive-never-capture. No page bytes are reproduced here. A source-tagged table existing on a branch does not prove every baseline consumer uses it. The coverage strategy explicitly separates such branches from the baseline. [11] [40]

INFERRED. A reduced mode should pass useful applications with withheld optional features. Booting alone does not establish the fallback. Likewise, a native reference capture establishes what one driver on one die accepted; it does not prove an unchanged answer is valid for every architecture, SKU or driver branch. [40]

6.10 The host is a victim, and quotas are not isolation

DOC. The GPU MMU, channel privilege, Kayfabe's checks and the trusted host driver form the address boundary. There is no independent security audit. Confining the whole VMM and using separate Unix credentials are deployment requirements/recommendations, not consequences of an ordinary host request being non-privileged. [9] [5]

UNMEASURED. A guest that faults a channel, hangs shared execution or occupies the host driver can affect the host's display session. The indexed evidence does not bound that effect. A useful victim keeps a live host context, data and UI workload across the event; starting a new test after recovery is weaker. The test also needs an equivalent native-client control and host responsiveness measurements, not just guest error logs. [9] [4]

DOC. Engine-wide notifications can wake unrelated registered clients. The project accepts spurious interrupt work and timing leakage rather than dropping a real completion wake. That does not establish a useful bound under an interrupt-heavy neighbour. The dated security review also lists incomplete limits for GPU time, encoder sessions, host apertures and driver request rate; it has not been repeated at this pin. [5] [9]

INFERRED. A fixed framebuffer budget can bound one allocation without guaranteeing fair scheduling or limiting a request stream. Per-VM video-memory, scheduling and call-rate policies need explicit service guarantees and adversarial tests. The hardware privilege boundary must

also be tested separately from Unix credentials and host request permissions.

6.11 Lifecycle, maintenance and unmeasured scope

Several GPUs. REPORTED. A Linux thin guest used two distinct GPUs, separately and concurrently. CUDA/UVM across devices, peer access and a Windows display device choice are outside that result. [15] [13]

Suspend and reset. REPORTED. Linux S3 passed three cycles in one boot at `559db30c`, on a physical RTX 4070 with host 595.91.07 and guest 580.159.04; Windows wake failures remain documented. Reboot scrubbing, driver reset and unload/reload are different lifetimes. No cold-boot pass proves arbitrary repeated reload or kill-during-work recovery. [46] [45] [44]

Other hosts. DOC. QEMU is the implemented VMM adapter. A cross-build does not test another CPU's memory mapping, interrupt or reset semantics. Other VMMs and arm64 require real adapter evidence. [13] [11]

Windows breadth. UNVERIFIED. The Linux Blackwell cell does not qualify Windows. No complete Windows Blackwell result at the pin is indexed here. Multi-head/hot-plug and display continuity across repeated reboots also need their own qualification. [11] [44]

Snapshot and migration. UNVERIFIED. This paper establishes no live snapshot/restore or VM-migration contract for in-flight GPU state. Preserve this as out of scope until a protocol accounts for device state, host objects, outstanding work and destination identity.

Driver-release cost. UNMEASURED. The driver matrix shows tested compatibility cells, not a ledger of engineering time per release. This paper supplies no estimate of how often layouts change, how much work each release costs, or what share of constants still needs hand maintenance. Those require a change/cost record of their own. [39]

6.12 Performance evidence must not move with the prose

DOC. The Linux baseline document explains why an otherwise attractive older comparison could not run with its host driver. A later usable comparison already contained Windows work. Its before/after delta therefore cannot price all Windows overhead. [57]

The distances in Section 5.5 are a more general warning: even a merged result can be hundreds of product paths away from the paper's subject. A repeated timing sample is not another boot, another die or another host. A ratio from nested KVM is a result for that configuration, and the control needs its own spread. Missing and failed samples must remain in the denominator. [16] [57]

6.13 Licences and release are not settled by a running VM

DOC. The source offers Apache-2.0 OR GPL-2.0-or-later with exceptions. Some nvproxy-derived files remain Apache-only, and the credits say the linked QEMU binary is not yet distributable under the GPL. The broker has a separate notice; vendor drivers and OS binaries are not supplied by Kayfabe. These are repository notices, not a legal opinion. [8] [6] [7]

DOC. NVIDIA's current driver agreement contains platform-use conditions and restrictions concerning commercial hosting, technical restrictions, and GeForce/Titan datacentre deployment. Their applicability depends on the actual software, hardware, agreement and deployment. This paper determines neither that every VM use is prohibited nor that an open-source emulator grants permission. Calling a project an alternative to vendor vGPU licensing does not settle whether any restriction is bypassed. A release needs qualified review of the actual arrangement. [79]

DOC. The public provenance policy is draft wording awaiting review. It does not claim a clean-room process and does not permit closed-binary material as public implementation data. Guest event tracing, such as Windows' own scheduling events, is distinct from analysing a closed binary; neither a private analysis memo nor an unreviewed summary becomes evidence by calling it a trace. No such findings or values are included here. [41] [68]

6.14 Test independence and the research stack

DOC. The predecessor records specific instrument failures: a configuration the mutation tool never read, a test baseline that omitted relevant suites, a count substituted for workload content, and an independent-looking tracker with the same missing transition as production. Those incidents justify planted failures and independent invariants. They do not establish that today's tests are unsound, or that multiple reviewers make an oracle independent. [16]

UNVERIFIED. This paper has not audited oracle independence for all current suites. The implementation triplets name what each test could miss, and the research questions ask what would expose a correlated model error. A green test on an unexercised path remains weak regardless of who wrote the test. [2] [48]

The full PR and historical-gap inventories are in the companion index. The following ten proposals bear directly on this argument; they are open research/integration work, not ten new verified product features. Their current base branches and heads are in that snapshot. The companion also separates the older stacks from the newer next-branch policy.

PR	Claim or proposed change	Question it bears on
140	Retain backing until invalidation acknowledges	Q2: retirement event
146	Preserve sparse mapping semantics	Q25: finite sparse coverage
99	Combined descriptor/mapping candidate	Q15: barrier cost and integration
169	Diagnose a consumed completion wake	Q24: packet/notification identity
82	Expose invalidate snapshot drift	Q17: concurrent table changes
75	Root-move/withdrawal contracts	Q26: lifetime identity
73	Native privilege stamps for kernel authority	Q36: privilege evidence
74	Split all-space walks at the ABI boundary	Q15: workload-dependent cost
19	Require a fresh GPU-walk completion event	Q23: fresh completion observation
88	Bound trap-side doorbell batches	Q15: who waits behind work

Their falsifiers are not substitutes for exact final-tree acceptance. A passing parent and a passing branch elsewhere do not qualify a merge. The project's batch policy makes that boundary explicit; refreshing an inventory does not perform the integration review. [2]

7 Thirty-six standalone attack questions

Each setup describes the system needed for the experiment. These are questions for public interface knowledge or a self-contained reproducer, not instructions to inspect a private trace or a repository-specific state machine. Proposed observations are not claimed results.

Q1. Must a small edit require a whole-space scan?

Setup: An emulator cannot intercept stores to guest GPU page tables. After a translation barrier it must discover changes before dependent commands run, although only a small range may have changed.

Observation to obtain: Identify a public changed-range certificate or bounded validation rule, then try a root or subtree change outside its claimed scope.

Decision it changes: A certificate that survives that counterexample permits incremental work. Otherwise the whole-space scan remains part of this design's correctness cost.

Q2. Which acknowledgement makes old backing reusable?

Setup: GPU work may still refer to a VA while software removes that host mapping, invalidates translation caches and wants to reuse its physical backing.

Observation to obtain: State the public host-driver ordering guarantee, or run a counter test that observes an old access after the proposed retirement event.

Decision it changes: A weaker guarantee requires a separate execution/lifetime fence; a sufficient one can simplify retirement without relying on elapsed time.

Q3. Does a longer observation window show late work or replacement work?

Setup: A fixed Windows graphics workload is run with default recovery settings and a longer diagnostic window. Host submissions and guest recovery each carry a lifetime identity.

Observation to obtain: Determine whether the original operation eventually completes, is canceled, or is replaced by new work after recovery. Locate the scheduler preemption request too.

Decision it changes: Only identified original completion supports a lateness explanation. Replacement or missing identity keeps the lost-completion explanation open.

Q4. Can live sparse reservations be combined without a hole?

Setup: Two adjacent GPU VA reservations intentionally absorb unbacked accesses. Software wants one larger reservation without changing any currently accessible address or pausing execution.

Observation to obtain: Find a public atomic replacement operation, or demonstrate a transition that avoids release-before-reserve faults under concurrent accesses.

Decision it changes: A valid primitive permits live geometry compression. Without it, empty-reservation recycling does not justify merging active sparse coverage.

Q5. What bounds a single host-driver call?

Setup: A software budget can stop starting operations, but cannot interrupt an already running native memory-reservation, map or invalidate call.

Observation to obtain: Measure per-call tails under idle and loaded conditions and identify any public bound, cancellation mechanism or asynchronous alternative.

Decision it changes: A usable bound enables deadline reasoning. Otherwise dependent work must tolerate an unbounded tail or the dependency must be moved ahead of the guest-visible barrier.

Q6. What must a lock holder promise before a worker parks?

Setup: A worker consumes a notification, fails a try-lock, and prepares to sleep while work remains. The holder may release before, during or after waiter registration.

Observation to obtain: Provide a register/recheck/release protocol and enumerate all three orderings, including retirement and reuse of the protected object.

Decision it changes: A missed final wake changes the handoff protocol. A correct protocol permits sleep without using a periodic timer as the actual source of progress.

Q7. What separates GPU clients running at different Unix UIDs?

Setup: Two VMM processes have distinct unprivileged Unix identities, each holding a host GPU driver client and USER-privilege channels in separate VA spaces.

Observation to obtain: Document or test which cross-client operations remain possible without explicit resource sharing, including direct naming and resource duplication.

Decision it changes: This determines what the credential policy actually closes and which operations still need application-level ownership checks or a stricter deployment boundary.

Q8. How much of a nested result belongs to nesting?

Setup: The same GPU workload can run natively, in a guest on a physical host, or in a guest whose host is itself a VM. Submission count, placement and driver versions can be held fixed.

Observation to obtain: Measure exit counts, per-exit time, computation time and output in all three configurations while recording clock state and host load.

Decision it changes: The decomposition tells which overhead is an emulator mechanism and which ratio cannot be transferred from a rented nested host to a physical desktop.

Q9. Which oracle distinguishes correct GPU execution from a plausible PASS?

Setup: A harness receives an application exit code, output bytes and an activity signal. A process can exit zero without checking a failed GPU synchronisation.

Observation to obtain: Inject separately a missing launch, wrong output, forced GPU error and CPU fallback; determine which independent observation rejects each while accepting the native control.

Decision it changes: This selects the evidence needed for the workload claim, rather than assuming either an activity counter or output equality alone proves the intended execution path.

Q10. What can a rented-machine GPU gate establish?

Setup: A provider operator can inspect approved source and possesses one run-scoped reporting capability. It can fabricate logs and a correctly bound structured result.

Observation to obtain: Separate errors a normal run can expose from claims requiring an observer or builder outside that operator's control.

Decision it changes: The answer defines the useful regression tripwire and prevents it from being treated as release provenance or an execution attestation.

Q11. What happens to the host desktop when a guest fails?

Setup: A host display session and a guest use one physical GPU. The host retains a live rendering context and reference data while the guest faults, hangs work or triggers recovery.

Observation to obtain: Measure host frame latency, output integrity, context survival and recovery; compare the same disturbance generated by an ordinary native host client.

Decision it changes: A host-wide failure restricts sharing claims even if guest addresses are isolated. Containment supports only the exercised fault classes and measured service bounds.

Q12. Could a mapping barrier acknowledge before host apply?

Setup: A guest edits GPU page tables, asks for an invalidate, and expects subsequent work to see the edit. The emulator still has an old host mapping and no universal fault/replay service.

Observation to obtain: Construct either a safe early-ack protocol or a stale-access counterexample, distinguishing an absent mapping that faults from a still-valid old mapping that returns wrong data.

Decision it changes: This decides whether blocking apply is inherent in the current authority model, or whether a separately proven fault/retry or versioning mechanism could remove it.

Q13. How should driver-release maintenance cost be measured?

Setup: An emulator serves an unmodified guest driver while independently selecting host-driver ABIs and die capabilities. New releases can change layouts, requests and behaviour.

Observation to obtain: For a sequence of public driver releases, record changed interfaces, implementation and test effort, unavailable hardware cells and remaining hand-maintained answers.

Decision it changes: The resulting ledger can price a supported release window and proactive measurements; a compatibility matrix alone cannot estimate that maintenance burden.

Q14. Which software agreements apply to this deployment?

Setup: A source-built emulator runs a vendor driver on GeForce-class hardware, possibly on rented infrastructure or as a commercial service; its own source has separate licence exceptions.

Observation to obtain: Obtain qualified review of the actual driver agreement, platform/hosting/datacentre terms, technical restrictions and combined-binary distribution permissions for that arrangement.

Decision it changes: The review determines release and deployment scope. Neither a working VM nor absence of a vendor vGPU component answers the contractual questions.

Q15. What is the worst barrier cost the guest can cause?

Setup: One translation barrier triggers a table traversal plus host maps, reservation changes, repair and invalidation. A guest can choose a fragmented layout within finite object caps.

Observation to obtain: Bound or measure each non-overlapping stage as a function of visited tables and native operations, then identify every host or guest client waiting behind the operation.

Decision it changes: This decides whether workload caps imply a useful latency/service bound or merely finite memory use with a potentially unacceptable shared delay.

Q16. Could isolated hardware translation remove the software walk?

Setup: Instead of reconstructing host GPU tables, imagine a host interface that accepts a guest translation root in a hardware-isolated context. The guest can mutate or recycle it.

Observation to obtain: Identify a public interface and prove ownership, privilege, revocation and concurrent-update semantics, including all privileged addressing paths.

Decision it changes: A usable interface changes the architecture. A root-validation check without ongoing isolation is not enough to replace mirroring safely.

Q17. What can a walk promise while tables change concurrently?

Setup: A GPU traversal reads a directory, then the guest replaces or recycles a lower table before the traversal reaches it. Unchanged addresses are supposed to remain usable.

Observation to obtain: Give a protocol and counterexamples for allowed old/new/fault outcomes, root generation reuse, partial reads and a changed subtree shared by two roots.

Decision it changes: This determines whether live walking needs validation, versioning or a stronger contract; treating every transient decode failure as a stable malformed mapping is insufficient.

Q18. How much coupling is inside the shared host driver?

Setup: Two clients use independent GPU VA spaces but issue memory-management calls through one host driver. One client repeatedly creates difficult reservation layouts.

Observation to obtain: Measure the other client's call latency and host-display latency, distinguishing per-client queues from any driver-wide serialisation and GPU execution contention.

Decision it changes: This determines whether per-space emulator gates isolate useful latency or leave a shared native bottleneck requiring admission or deployment limits.

Q19. Which limits give a VM a useful service guarantee?

Setup: A guest has a fixed video-memory allocation but shares GPU scheduling, encoder capacity and host-driver request service with a live host workload.

Observation to obtain: Vary allocations, scheduling requests and request rate independently; measure the victim's worst service gap and whether releasing the load restores service without a reset.

Decision it changes: This separates memory isolation from fairness and identifies the quota or admission contract required for a stated responsiveness target.

Q20. What state must a GPU VM snapshot or migration contain?

Setup: A VM is stopped with GPU work in flight, host-owned memory objects and queued completions. It may resume in another process or on a different compatible GPU.

Observation to obtain: Specify which state can be drained, reconstructed or transferred and how old completions, host identities and vendor-driver state are prevented from crossing lifetimes.

Decision it changes: This identifies a supportable snapshot/migration mode or makes its exclusion explicit, rather than assuming CPU-memory snapshot semantics cover the device.

Q21. Can unaffected ranges submit through a refresh?

Setup: One host VA space runs work on range A while the guest changes disjoint range B. A global gate currently withholds all new submissions into that space.

Observation to obtain: Provide a bounded range-dependency certificate that survives root changes, aliases, publication races and a full change-history buffer.

Decision it changes: It permits narrower admission if sound; otherwise the implementation must keep conservative holds and reduce the work behind them.

Q22. What preserves a running command ring across relocation?

Setup: A stock driver requests a new GPU command-reference ring while commands from the old ring are in flight. Software wants native relocation rather than relaying every command.

Observation to obtain: Demonstrate cursor continuity, ownership of both ring memories, retirement of the old source and a completion produced only by the replacement ring.

Decision it changes: This selects between a supported native move, software mediation from birth and a separate cooperation tier without relying on an idle cursor as completion proof.

Q23. Which clock separates GPU completion from CPU observation?

Setup: Software timestamps a submission and its first read of a completed GPU-written marker. The observing worker may spend time in a native driver call.

Observation to obtain: Find an independent GPU-time witness and clock relation, or a second observer that distinguishes execution delay from delayed sampling without adding a correctness poll.

Decision it changes: This determines which latency component can be reported and which must remain an ambiguous submission-to-observation sum.

Q24. How can a notification be tied to one operation?

Setup: Several channels and clients share engine-wide completion notifications. A CPU may see an interrupt before it samples its own completion marker.

Observation to obtain: Specify the minimum identity, masking/coalescing state and ordering evidence needed to establish that a notification covered the particular operation.

Decision it changes: This chooses between a valid delivery-latency sample and an explicitly incomplete correlation; an unrelated interrupt must not manufacture a completed record.

Q25. What happens when a finite sparse budget is exhausted?

Setup: A guest intentionally maps unbacked zero/absorbing GPU pages. The host has a finite reservation budget and leaves additional requested coverage invalid on exhaustion.

Observation to obtain: Identify a recoverable driver-visible refusal point, or a workload that requires those semantics after that point and cannot recover from an invalid mapping.

Decision it changes: This defines whether the bounded mode is compatible, needs a new representation or must exclude that workload; counting the deficit alone does not choose among them.

Q26. Which identity survives root replacement and reset?

Setup: A guest VA-space object can replace its page-directory root, reuse an old root address and reset a channel while host mappings or completions remain outstanding.

Observation to obtain: Give an identity/epoch scheme and adversarial sequence that distinguishes same object, new root and new lifetime, including delayed callbacks.

Decision it changes: This determines when a cached mapping, source or completion can be reused and when old obligations must be retired or canceled.

Q27. Which layer fails on resume?

Setup: A guest resumes from S3 with the emulated GPU state retained, but platform timers and the operating system also need to recover.

Observation to obtain: Compare matched plain-VMM and device-enabled guests, then require post-wake content-checked GPU work and locate the first differing state or wait.

Decision it changes: A platform-only failure changes the harness diagnosis; a device-specific failure remains a device lifecycle defect even if its firmware-facing state machine is running.

Q28. What distinguishes an ARM port from a cross-build?

Setup: Logic code builds for arm64, while the host adapter must register shared GPU memory, handle different page sizes and deliver interrupts and resets.

Observation to obtain: Execute the required memory and lifecycle primitives on a VM-capable ARM host and identify which ordering or fault-resumption semantics differ from the x86 adapter.

Decision it changes: This determines whether an adapter capability is missing or whether ordinary porting work is sufficient. Compilation alone does not choose between them.

Q29. What must a second VMM adapter guarantee?

Setup: A GPU device core needs bounded traps, shared-memory views, notification and distinct reset/lifetime events. It currently runs inside one VMM process.

Observation to obtain: Express those guarantees using another public VMM/device protocol, then identify any operation whose latency or semantics cannot satisfy the same contract.

Decision it changes: This prices real hypervisor independence and exposes requirements hidden by the first adapter, without requiring the reader to know its internal traits.

Q30. How should simultaneous host/guest use be graded?

Setup: A screenshot shows a host desktop and a guest both using one GPU. The host can also run a repeatable render or compute workload with an independent output check.

Observation to obtain: Measure each alone and together, preserving clocks, settings and content; report throughput and tail frame/completion gaps for both sides over repeated boots.

Decision it changes: This replaces visual coexistence with a bounded shared-use claim and shows whether the advertised benefit costs unacceptable host responsiveness.

Q31. Which driver/version cells are most discriminating?

Setup: Host and guest driver versions are independent. Public releases can change structures inside a branch and add fallback requests only on error paths.

Observation to obtain: Select cells using actual public interface changes and test that selection against normal and refusal-driven paths, rather than only major-version labels.

Decision it changes: This determines the supported window and where a decoder oracle is insufficient without a real workload and recovery test.

Q32. What is a usable mode for an unmeasured die?

Setup: An emulated GPU can withhold optional capabilities when their exact-die provenance is unavailable, while keeping the guest driver unmodified.

Observation to obtain: Identify a minimal public capability surface, exercise useful applications and recovery, and use wrong-answer mutations to show which advertised fields actually matter.

Decision it changes: This defines a working reduced mode rather than treating a named refusal or a boot as a complete compatibility result.

Q33. Which failures of a viewer may affect a guest?

Setup: A guest submits display flips while a separate viewer receives copied frames. The viewer can stall, restart, flood requests or retain its last frame slot indefinitely.

Observation to obtain: Specify and test which guest completions must continue, which frames may be dropped, and the bounded ownership of buffers across reconnect and reset.

Decision it changes: This separates guest display semantics from transport quality and determines where a viewer failure may legitimately cause degraded output rather than a guest hang.

Q34. Which independent invariant would expose a shared test-model error?

Setup: An event-driven queue implementation and its reference model were written from one informal progress story. Both could forget the same transition.

Observation to obtain: Derive a conservation rule from externally visible submissions, cancellations and effects; plant a dropped final obligation and show the oracle rejects it.

Decision it changes: This tests oracle independence through a concrete counterexample, instead of assuming separate code or multiple reviewers imply separate reasoning.

Q35. Which repetitions estimate the user-visible failure?

Setup: Iterations in one boot are stable, but boot state, host placement and concurrent load can change. Timeouts and failed boots have no final throughput sample.

Observation to obtain: Separate within-boot, between-boot and between-host spread while retaining failures and missing samples in the denominator.

Decision it changes: This chooses an acceptance sample unit and prevents a favourable conditional median from representing a configuration that users cannot reliably reach.

Q36. What does a GPU USER-privilege test actually prove?

Setup: A host client can create channels with a hardware privilege level, but a guest also names objects and supplies addresses through a software decoder.

Observation to obtain: Test physical and virtual operands on each relevant engine and trace the trusted allocation fact authorising privileged work, independently of guest handles and Unix UID.

Decision it changes: This establishes the scope of the hardware boundary and the remaining software authority checks; one accepted ordinary request cannot prove all three permissions.

A Method and reproducible commands

This edition used source reads, Git comparisons, publication checks and XeLaTeX. It ran no Cargo or Clippy command, VM, real-GPU gate, provider operation or remote-host experiment. Every hardware result is carried from a dated source. Publication checks here do not turn those records into measurements of the cover revision.

A.1 Sources and code distance

`sources_v3.json` records repository revisions, paths and Git blobs, plus the three external primary sources checked on the publication date. Numeric citations link directly to those sources. The full mapping, historical evidence names, PR metadata and gap topics are in the companion index, `kayfabe_v3_index.md`. That branch link moves with publication; the source revisions recorded inside it do not. External pages can change after checking.

The following reconstructs repository-source identity and the result-distance calculation:

```
python3 - <<'PY'
import json, subprocess
from pathlib import Path
p = Path('docs/whitepaper/sources_v3.json')
for row in json.loads(p.read_text())['sources']:
    ref = row['revision'] + ':' + row['path']
    blob = subprocess.check_output(['git', 'rev-parse', ref], text=True)
    assert blob.strip() == row['blob'], ref
PY
# PIN is the cover revision; RESULT is a runtime from the results table.
git diff --name-only RESULT PIN -- crates qemu cuda
git rev-list --left-right --count PIN...RESULT
```

Changed paths include additions, removals and modifications. Commit counts cover the whole Git graph, not only the three product directories. Zero result-only commits means

ancestor; it is not a passing-test status. A divergent result has commits absent from the pin. `result_distances_v3.json` preserves full runtime hashes and rerun status. No result in that file was rerun on hardware for this edition.

A.2 PR snapshot and historical gap index

```
gh pr list --state open --limit 200 \
  --json number,title,headRefName,headRefOid,baseRefName,isDraft,url
gh pr list --state all --limit 250 \
  --json number,headRefName,baseRefName,state,url
```

The snapshot was refreshed after the revision work. It freezes each open PR's number, base, head, draft state and known parent PR: a parent is identified by an exact base-branch to PR-head-branch match, including merged parents. A branch base without such a match remains unknown. A recorded parent is metadata, not proof that the full dependency was reviewed. Live heads, draft states and bases will change after capture.

The companion also preserves 101 historical gap topics. They include closed and superseded rows; the count is not a current defect count. Application gaps, capability gaps and postponed features are identified separately there. No template inventory occupies the main argument.

A.3 Build and verify the publication

```
python3 -m unittest discover -s docs/whitepaper -p test_build_v3.py
python3 scripts/ci/debt.py claims
git diff --check
# Commit TeX inputs before producing a publishable PDF.
bash docs/whitepaper/build.sh v3
python3 docs/whitepaper/build_v3.py --check
pdfinfo docs/whitepaper/kayfabe_v3.pdf
pdftotext -layout docs/whitepaper/kayfabe_v3.pdf -
```

CODE. The v3 target runs XeLaTeX twice with halt-on-error and an input recorder. It checks for TeX errors, missing glyphs and undefined references. Recorded repository inputs must match committed blobs. A JSON receipt binds input hashes, source commit, compiler and output digest. A later PDF-only commit leaves the input blobs unchanged. This is a local publication check, not a signed attestation of a trusted compiler or host.

A.4 Reproducing hardware evidence is separate work

The following are repository entry points, not commands run for this paper. They require an authorised, exclusively leased test machine and configuration capture. [2]

```
bash scripts/bench/v3_gates.sh
KF_DEVICE=kf3 bash scripts/fastguest/fast_suite.sh <tag> 180
bash scripts/bench/cuda_ladder.sh host <tag>
bash scripts/bench/cuda_ladder.sh guest <tag>
bash scripts/bench/perf_compare.sh <rev> \
  --baseline <rev-or-result-dir> --lanes bare,fast --reps 3
```

DOC. The performance method requires source revision, actual die, drivers, nested status, placement, settings, flags, sample unit, failed runs and the control's spread. Changing a paper's source pin does not update a hardware receipt. [57]

For the stall case, preregister completion, recovery and latency outcomes separately. Keep default-watchdog and longer-observation arms. Missing scheduler preemption timestamps, packet association, native GPU clocks or guest events remain missing observations. A capped trace ring cannot silently shrink a failure denominator. [68] [78]

B Glossary and historical names

ABI Application binary interface: layouts and calling conventions.

BAR
PCI address window. Only BAR0 writes trap in this design.

CE / GR
Copy engine / graphics-compute engine in the source records.

GPFIFO / GP / pushbuffer
Ring of command references / one reference / command bytes.

GET / PUT
Fetch cursor / publication cursor; fetching does not establish execution.

IOVA / GPA
Device-visible I/O virtual address / guest physical address.

IRQ / MSI / DPC
Interrupt request / message-signalled interrupt / deferred Windows work.

MEM_OP / split
Memory-operation command / ordering boundary for a required map refresh.

PTE / PDE / PDB
Page-table entry / page-directory entry / page-directory base.

RM / GSP
Resource manager / GPU System Processor.

TSG / USERD
Timeslice group / channel producer-consumer state memory.

USER
GPU channel privilege; distinct from host request permission and Unix credentials.

UVM
Unified virtual memory, including fault delivery and replay.

WDDM / TDR
Windows Display Driver Model / timeout detection and recovery.

ogkm
NVIDIA's open GPU kernel modules, used as pinned reference sources.

B.1 Decoding old file names

These are lookup aids for historical sources, not extra concepts required by the argument.

v3 / pre-v3
Current `kf-*` product / frozen `kayfabe-*` predecessor.

M / B-prime / baton
Alternative relay-handoff designs. [61]

ENS5 / ENS6 / ENS7
: Sparse-admission undo / host-call churn and slices / framebuffer-scaled sparse budgets. [62] [63] [64]

ENS8 / ENS10 / ENS11
: Consumed-ring wakes / mapping scope / split-latency continuation. [48] [67] [68]

T-space
: Address space reserved for privileged rewritten work. [13]

Boot labels and candidate numbers
: Harness identifiers, not release versions or grades. The companion records the few identifiers needed to find the unverified analysis.

C Change record

The predecessor describes the earlier multi-process architecture and remains superseded. The first v3 edition replaced that model, pinned the source, introduced explicit limitations and added the committed-input publication gate. [16] [1]

This cold-reader revision states the stall hypothesis before the internals, distinguishes CODE from DOC and REPORTED from UNVERIFIED, restores positive Windows results and native performance denominators, and makes host-plus-guest performance an explicit missing result. It replaces generic concept boxes with four mechanism diagrams plus the lifecycle diagram. The full inventories move to the companion; 36 questions now state their own experimental setup and decision. Neither this revision nor its Git-distance table qualifies new hardware.